

Fast and Energy-Efficient Pipelined Vedic Multiplier Design for Modern Cryptographic Processors

Akkapolu Sankararao, Naveen Kollepara, Vashist Managari, Binsu J Kailath

Department of Electronics and Communication Engineering

IIITDM Kancheepuram, Chennai, Tamil Nadu, 600127, India

Email: ec23d0008@iiitdm.ac.in, ec21b1030@iiitdm.ac.in, ec21b1038@iiitdm.ac.in, bkailath@iiitdm.ac.in

Abstract—Efficient multiplier design is critical in modern cryptographic processors, where repeated large-number arithmetic is fundamental to public-key schemes like Rivest-Shamir-Adleman (RSA). This paper proposes a pipelined Vedic multiplier architecture optimized for energy-efficient and high-performance modular operations. Leveraging the Urdhva Tiryakbhayam (UT) sutra, the design enables parallel partial product computation, reduced carry propagation, and minimal interconnect complexity. The multiplier was implemented and evaluated within an RSA cryptographic framework using both FPGA (Zynq UltraScale+ ZCUI 04) and ASIC (32nm RVT technology) platforms. Comparative analysis with state-of-the-art designs demonstrates significant improvements of upto 74.08% area savings, 91.25% power reduction, and 92.01% reduction in computational delay compared to state-of-art designs. Simulation results and real-time verification confirm the functional correctness and superior performance of the proposed architecture.

Index Terms—Vedic multiplier, pipelining, data security, cryptography, RSA, encryption, decryption, public key, 64-bit pipelined floating point, FPGA.

I. INTRODUCTION

Multiplication is a fundamental operation in modern computing systems, playing a vital role in the hardware implementation of cryptographic, digital signal processing, artificial intelligence, and machine learning algorithms. In cryptographic systems such as RSA and AES, multipliers are required to efficiently perform modular arithmetic [1], [2]. For such applications, it is essential that multiplier architectures support modular configurability while also achieving high speed, low power consumption, and reduced silicon area to meet performance and resource constraints in secure computing environments [2].

Vedic multipliers, particularly those based on the Urdhva Tiryakbhayam (UT) sutra [3], provide an efficient approach for hardware multiplication by enabling the parallel generation of partial products (PP's). This parallelism supports pipelining, allowing all bit positions to be computed simultaneously without waiting for lower-order products, unlike conventional sequential methods [4]–[6]. Additionally, the simplicity of binary bit-wise operations further enhances speed and design modularity. These features make UT-based multipliers highly suitable for high-performance cryptographic systems [7], [8] where speed, parallelism, and configurability are essential. The UT sutra has inspired a variety of hardware-oriented multiplier architectures aimed at improving speed, power, and area efficiency in digital systems.

Prior research has introduced UT-based variations such as recursive architectures for large operands, UT combined with carry-save adders (CSA) [9], and compressor-based UT designs [3]. Other advancements include FPGA implementation of parallelization in multiplication for crypto-systems [10], and UT combined with Booth encoding to reduce partial products [4]. While pipelined architectures have demonstrated improvements in speed, power, or area individually, each approach involves certain trade-offs. Some designs optimize delay at the cost of increased area, while others achieve low power but involve complex routing and logic overhead in cryptographic hardware [6], [11]. To address these limitations, this work presents an optimized pipelined UT-based multiplier that balances energy efficiency, computational speed and area, making it suitable for high-performance cryptographic processors. The proposed design achieves up to 95.49% improvement in computational efficiency, 91.25% reduction in power consumption, and 74.08% area savings compared to state-of-art implementations.

The remainder of this paper is organized as follows. Section II describes the proposed pipelined Vedic multiplier architecture. Section III details the implementation of the RSA algorithm using the proposed multiplier. Section IV discusses FPGA and ASIC-based evaluation and performance metrics. Finally, Section V concludes the paper with key findings.

II. ENHANCED PIPELINED VEDIC MULTIPLIER DESIGN

Unlike traditional sequential multiplication techniques [11], [12], we propose a pipelined architecture which leverages the inherent parallelism of the Vedic UT Sutra to decompose the multiplication process into multiple concurrent stages. This parallel PP generation significantly reduces the number of sequential operations, thereby minimizing critical path delay, eliminating idle cycles. The design's modularity also enables scalability across varying bit-widths, making it highly suitable for high-performance and low-power applications.

A. Mathematical Methodology

The architecture of the proposed pipelined Vedic multiplier is shown in Fig. 1. It consists a two-stage pipeline implementation, which is explained in the following steps:

1) **Vedic Partial Product (PP) Generator**: This block uses the UT method to generate partial products (PP00, PP01,

PP10, etc.) in parallel, based on vertical and crosswise multiplication [3].

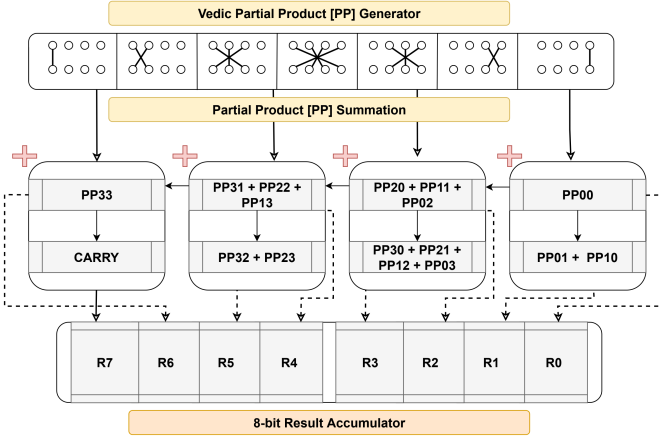


Fig. 1. Architecture of the proposed pipelined Vedic multiplier.

2) **Partial Product (PP) Summation:** The generated partial products are grouped and added in parallel through multiple pipeline stages. Each block sums a set of partial products concurrently (e.g., PP20 + PP11 + PP02). This approach avoids sequential addition and enables overlapping execution. Carry handling is managed in the final stage (PP33 + Carry), preparing the result for accumulation for storing in an accumulator register.

Let two 64-bit operands A and B be expressed as:

$$A = \sum_{i=0}^{63} A_i \cdot 2^i, \quad B = \sum_{i=0}^{63} B_i \cdot 2^i \quad (1)$$

The multiplication result based on UT is computed using:

$$C = \sum_{i=0}^{63} \sum_{j=0}^{63} PP_{i,j} \cdot 2^{i+j} \quad \text{where} \quad PP_{i,j} = A_i \cdot B_j \quad (2)$$

The design also supports floating-point numbers. Decimal inputs are converted to 64-bit IEEE-754 format to ensure compatibility and precision. The converted operands are fed into the 64x64 Vedic multiplier. Intermediate results are processed using a carry-save adder to reduce addition latency.

III. ADVANTAGE IN IMPLEMENTING RSA ALGORITHM WITH VEDIC MULTIPLIER

The RSA algorithm [2] implemented as shown in Fig. 2, relies on modular exponentiation, which requires repeated large integer multiplications [1]. As shown in Fig. 3, the number of multiplications increases rapidly with message bit-width, especially during decryption. This makes multiplication the dominant computational workload.

The proposed pipelined Vedic multiplier offers a hardware-efficient solution that accelerates RSA operations while reducing power and area overhead. Its pipelined structure and parallel partial product generation deliver faster computation with lower energy consumption. These characteristics are particularly beneficial for embedded and secure hardware modules.

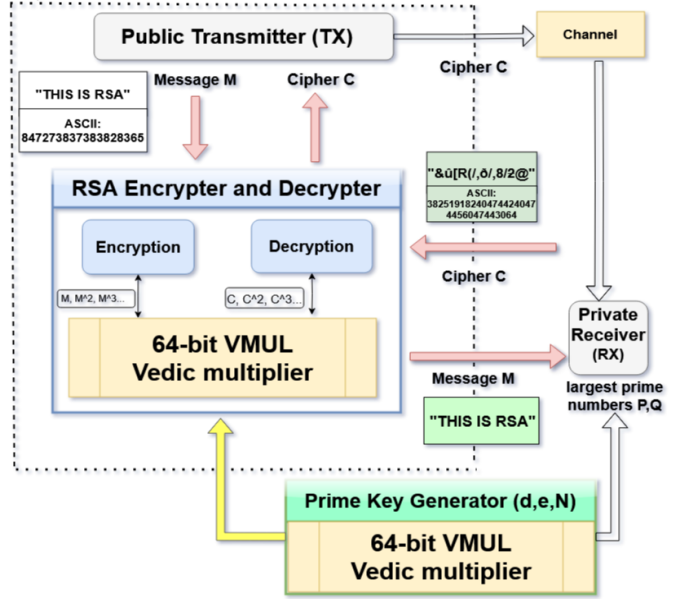


Fig. 2. Block diagram of RSA implementation using the proposed Vedic multiplier.

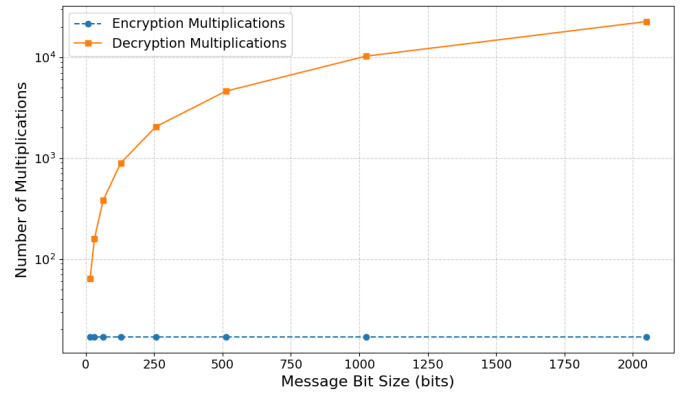


Fig. 3. Number of multiplication operations required for RSA encryption and decryption as message bit length increases.

A. Key Generation, Encryption and Decryption Process

RSA security is based on the difficulty of factoring large prime products. The RSA crypto-system involves two keys: a public key for encryption and a private key for decryption. Key generation includes:

- Selecting two large prime numbers p and q , computing modulus as $N = p \times q$ and calculate the Euler's function:

$$\varphi(N) = (p-1)(q-1) \quad (3)$$

- Choosing public exponent e such that

$$1 < e < \varphi(N) \quad (4)$$

$$\gcd(e, \varphi(N)) = 1 \quad (5)$$

- Computing the private exponent d as:

$$d = e^{-1} \mod \varphi(N) \quad (6)$$

The public key consists of (N, e) , while the private key is (N, d) . To transmit a message M , the sender encrypts it using the receiver's public key to obtain the cipher C . The $vmul[\cdot]$ denotes Vedic multiplication.

$$C = vmul[M^e \cdot mod N] \quad (7)$$

Similarly, the receiver decrypts the cipher C , forming the decrypted message M_{dec} using the private key (N, d) , only available to the receiver, making the message M immune to hacking or other attacks.

$$M_{dec} = vmul[C^d \cdot mod N] \quad (8)$$

The key part is the exponential products, M^e, C^d which are used extensively, are calculated using v to optimize this step in a state-machine implementation shown in equations 9. It calculates the exponent of a number M to the power e as continuous Vedic products for M, e times. These multiplications are also performed using $vmul$.

$$M^e = \prod_{i=1}^e M \quad C^d = \prod_{i=1}^d C \quad (9)$$

where e and d are given by Equations 3-6.

As illustrated in Fig. 3, both the encryption and decryption modules use the 64-bit Vedic multiplier, ensuring accelerated and efficient computation.

B. Use of Multiplier in Modular Exponentiation

As Fig. 3 shows, the number of multiplications required for decryption grows exponentially with the message size. In contrast, encryption typically uses a fixed exponent (commonly 65537). This shows the importance of an efficient multiplier to avoid performance bottlenecks in decryption [6]–[8].

IV. PERFORMANCE EVALUATION ON FPGA AND ASIC PLATFORMS

A. FPGA Implementation and Benchmarking

The design was implemented on a ZedBoard Zynq-7000 SoC using AMD Vivado 2022.2. The functional simulation of the proposed multiplier in Vivado yielded the waveform shown in Fig. 4, which confirms the correctness of the design. The entire RSA encryption and decryption flow was verified in simulation as shown in Figs. 5,6.

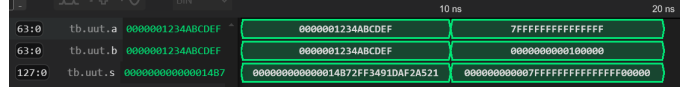


Fig. 4. FPGA simulation waveform of the proposed Vedic multiplier.

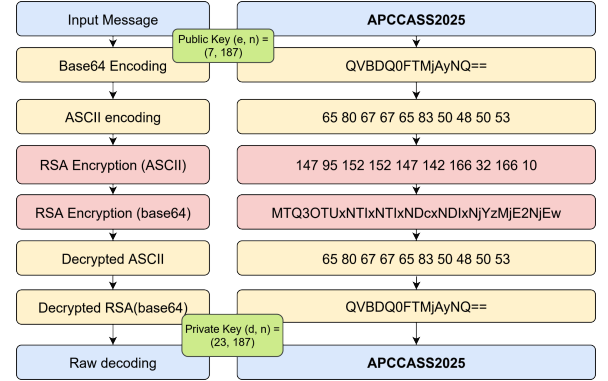


Fig. 5. Detailed encryption and decryption flow of the RSA algorithm using the proposed multiplier.

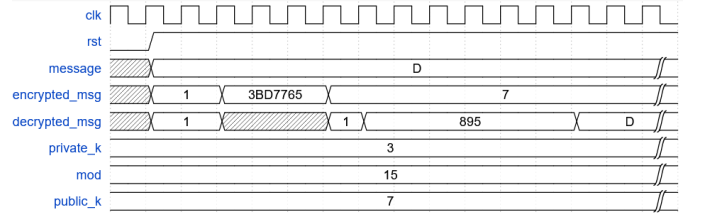


Fig. 6. FPGA simulation results showing end-to-end RSA encryption and decryption operation.

Table I compares the FPGA resource utilization and performance of the proposed design against state-of-the-art multipliers [3]–[6], [11], [12]. The design achieved a reduction of up to 92% in delay, a reduction in 72% in LUT usage, and significant power savings.

Table II shows the dynamic power consumption during RSA operations, where the design achieves up to 90% dynamic power reduction compared to Booth and Karatsuba multipliers. It is to note that the power compared in Table I is Total Power, that is dynamic and static combined. The power compared in Table II is only for Dynamic power.

TABLE I
COMPARISON OF FPGA RESOURCE UTILIZATION AND PERFORMANCE METRICS FOR DIFFERENT 64-BIT MULTIPLIER DESIGNS

Multiplier [64-bit]	XOR count	LUT	Delay (ns)	Power (mW)
Array multiplier	8000	12538	96.739	260
Wallace [5]	8112	8183	28.664	119
Karatsuba [6]	462	9103	7.51	187
Scaling based Vedic [3]	5365	10028	27.023	NA
Compressor based Vedic UT [3]	62	9401	13.766	154
Tree-based Vedic UT [11]	NA	2750	13.870	NA
CSA based Vedic UT [12]	NA	7342	11.48	128
Proposed Vedic UT Design	0	1504	6.5	103

TABLE II
COMPARISON OF FPGA IMPLEMENTATION RESULTS FOR RSA ENCRYPTION AND DECRYPTION USING VARIOUS STATE-OF-THE-ART MULTIPLIER ARCHITECTURES

RSA System (with Multiplier)	LUT	Flip-Flops	Dynamic Power (in mW)
Booth Multiplier [4]	8169	3230	51
Array Multiplier	4080	3166	24
Wallace [5]	6023	3369	31
Vedic UT Multiplier [3]	6213	3369	32
Karatsuba [6]	4379	3166	22
Proposed Design	2325	3143	6

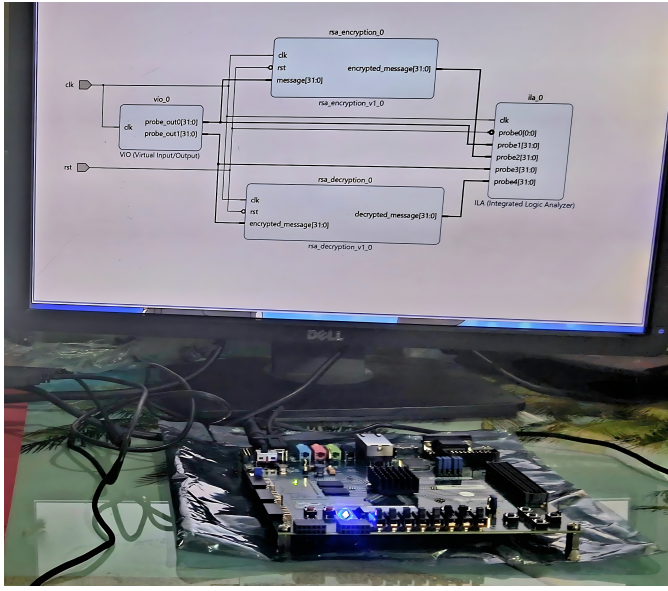


Fig. 7. FPGA schematic view with Integrated Logic Analyzer (ILA) and Virtual Input Output (VIO) IP cores.

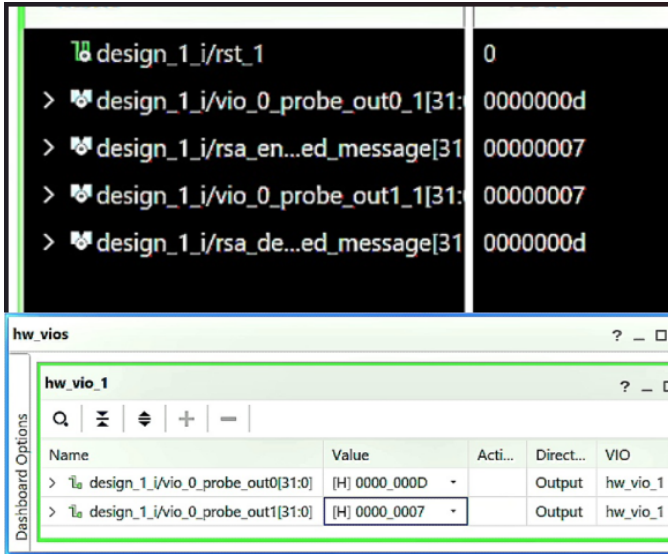


Fig. 8. FPGA implementation waveform of RSA encryption and decryption process viewed on a VIO.

For the hardware validation of the RSA system in the FPGA, Figs. 7, 8 shows the encryption and decryption modules along with two IPs, Integrated Logic Analyzer (ILA) and Virtual Input Output (VIO) used for monitoring internal signals in real time implemented in FPGA at 100MHz clock frequency.

B. ASIC Implementation and Area-Power Analysis

ASIC-level synthesis was performed using Synopsys Design Compiler Version T-2022.03-SP5 with a 32 nm RVT library. Fig. 9 shows that the proposed design achieved up to 74.08% area savings, 91.25% power reduction, and 95.49% improvement in computational delay compared to existing state-of-the-art implementations [3], [5], [6].

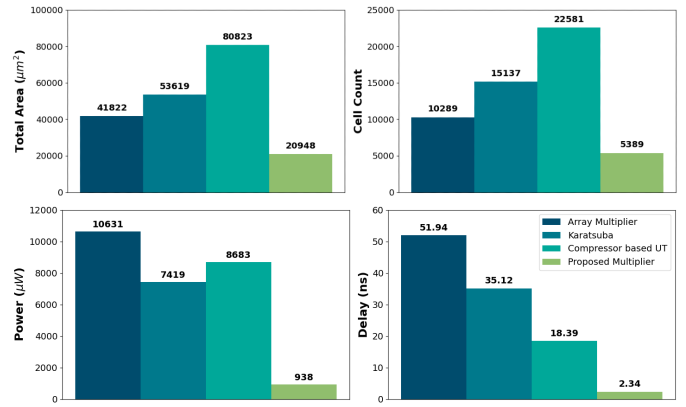


Fig. 9. ASIC synthesis results from Synopsys Design Compiler.

V. CONCLUSION

This paper presented an optimized pipelined Vedic multiplier for cryptographic applications, particularly RSA. By combining parallel partial product computation and deep pipelining, the design overcomes limitations of prior architectures in power, area, and delay [3]–[6], [11], [12]. FPGA implementation demonstrated up to 92% performance improvement in delay, 72% LUT reduction, and 90% dynamic power savings. ASIC synthesis further validated up to 74.08% area savings and 91.25% power reduction. These results confirm the design's suitability for secure, high-performance, and resource-constrained cryptographic systems.

REFERENCES

- [1] R. Imam, Q. M. Areeb, A. Alturki and F. Anwer, "Systematic and Critical Review of RSA Based Public Key Cryptographic Schemes: Past and Present Status," in IEEE Access, vol. 9, pp. 155949-155976, 2021, doi: 10.1109/ACCESS.2021.3129224.
- [2] R. L. Rivest, A. Shamir, and L. Adleman. 1978. A method for obtaining digital signatures and public-key cryptosystems. Commun. ACM 21, 2 (Feb. 1978), 120–126. <https://doi.org/10.1145/359340.359342>
- [3] Sujitha, S., Kailath, B. J. (2021). High speed power efficient Vedic arithmetic modules on Zedboard-Zynq-7000 FPGA. International Journal of Circuit Theory and Applications, 49(11), 3683–3718. <https://doi.org/10.1002/cta.3110>
- [4] Ghasemi MM, Fathi A, Mousazadeh M, Khoei A. A new high speed and low power decoder/encoder for Radix-4 Booth multiplier. Int J Circuit Theory Appl. 2021;1-15. <https://doi.org/10.1002/cta.2985>
- [5] Abed S, Mohd BJ, Al-bayati Z, Alouneh S. Low power Wallace multiplier design based on wide counters. Int J Circuit Theory Appl. 2012; 40(11):1175-1185. <https://doi.org/10.1002/cta.779>
- [6] Navarro-Torrero, P., Camacho-Ruiz, E., Martínez-Rodríguez, M. C., Brox, P. (2024). Design of a Karatsuba multiplier to accelerate digital signature schemes on embedded systems. 2024 IEEE Nordic Circuits and Systems Conference (NorCAS), 1–7. <https://doi.org/10.1109/NorCAS64408.2024.10752484>
- [7] B. Zhang, Z. Cheng and M. Pedram, "Design of a High-Performance Iterative Barrett Modular Multiplier for Crypto Systems," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 32, no. 5, pp. 897-910, May 2024, doi: 10.1109/TVLSI.2024.3368002.
- [8] S. Singh Dhanda et al., "A Fast and Efficient 191-bit Elliptic Curve Cryptographic Processor Using a Hybrid Karatsuba Multiplier for IoT Applications," in IEEE Access, vol. 12, pp. 144304-144315, 2024, doi: 10.1109/ACCESS.2024.3472650.
- [9] R. Raj, D. S and R. P, "Performance Analysis of 32-Bit Vedic Multipliers for Different Adder Configurations," 2022 IEEE 19th India Council International Conference (INDICON), Kochi, India, 2022, pp. 1-6, doi: 10.1109/INDICON56171.2022.10040134.

- [10] Li, B., Yan, Y., Wei, Y., Han, H. (2024). Scalable and parallel optimization of the number theoretic transform based on FPGA. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 32(2), 291-304. <https://doi.org/10.1109/TVLSI.2023.3312423>
- [11] A. Jais and P. Palsodkar, "Design and implementation of 64 bit multiplier using vedic algorithm," 2016 International Conference on Communication and Signal Processing (ICCSP), Melmaruvathur, India, 2016, pp. 0775-0779, doi: 10.1109/ICCSP.2016.7754250.
- [12] M. Barakat, W. Saad and M. Shokair, "Implementation of Efficient Multiplier for High Speed Applications Using FPGA," 2018 13th International Conference on Computer Engineering and Systems (ICCES), Cairo, Egypt, 2018, pp. 211-214, doi: 10.1109/ICCES.2018.8639254.
- [13] W. Tan, B. M. Case, A. Wang, S. Gao and Y. Lao, "High-Speed Modular Multiplier for Lattice-Based Cryptosystems," in *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 68, no. 8, pp. 2927-2931, Aug. 2021, doi: 10.1109/TCSII.2021.3064232.